



DQ800-Lab02- Programmability-Objects-SQL

Laboratorio 2 – DQ-800: Implement Programmability Objects with SQL

Módulo: DP-800

Práctica: Implement programmability objects with SQL

Base de datos: AdventureWorksLT

Fuente del enunciado: [Microsoft Learn — Write advanced T-SQL queries](#)

1. Introducción

 **[RELLENAR CON TUS PALABRAS: Explica brevemente el objetivo general de esta práctica: crear y utilizar objetos de programabilidad de SQL Server (vista, procedimiento almacenado, función escalar, función de tabla en línea y trigger) para centralizar lógica y mejorar la mantenibilidad, trabajando sobre la base de datos AdventureWorksLT]**

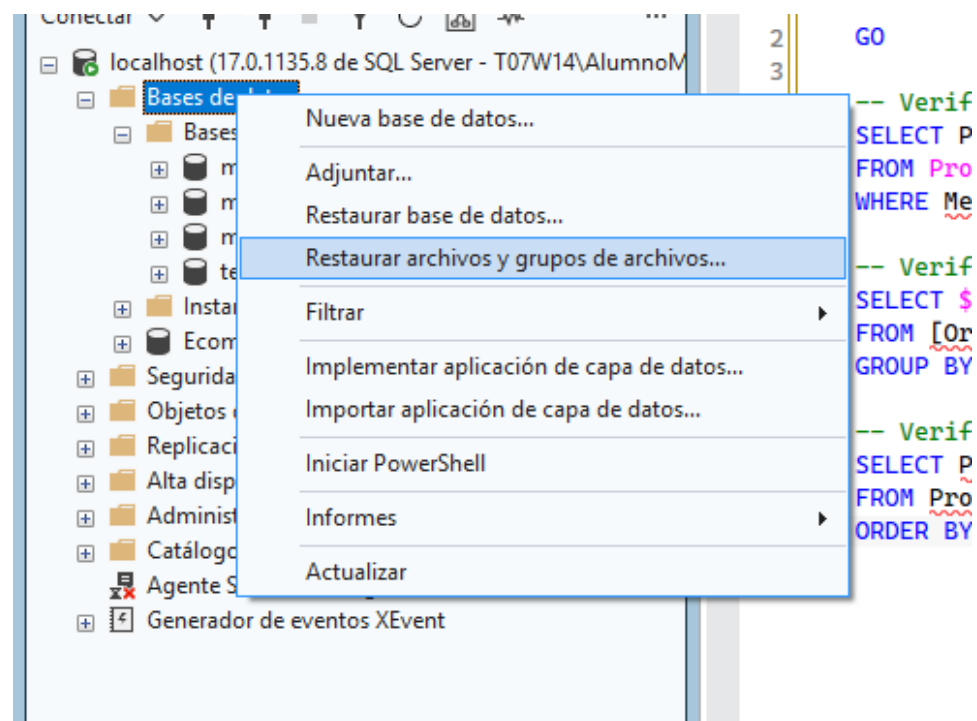
1.1 Objetivos

- Crear una vista para simplificar consultas complejas.
- Escribir un procedimiento almacenado que encapsule una operación de negocio.
- Implementar una función escalar para cálculos reutilizables.
- Construir una función de tabla en línea (TVF) para conjuntos de resultados parametrizados.
- Añadir un trigger que responda automáticamente a cambios de datos.

1.2 Requisitos previos

- SQL Server 2019+ o Azure SQL Database.

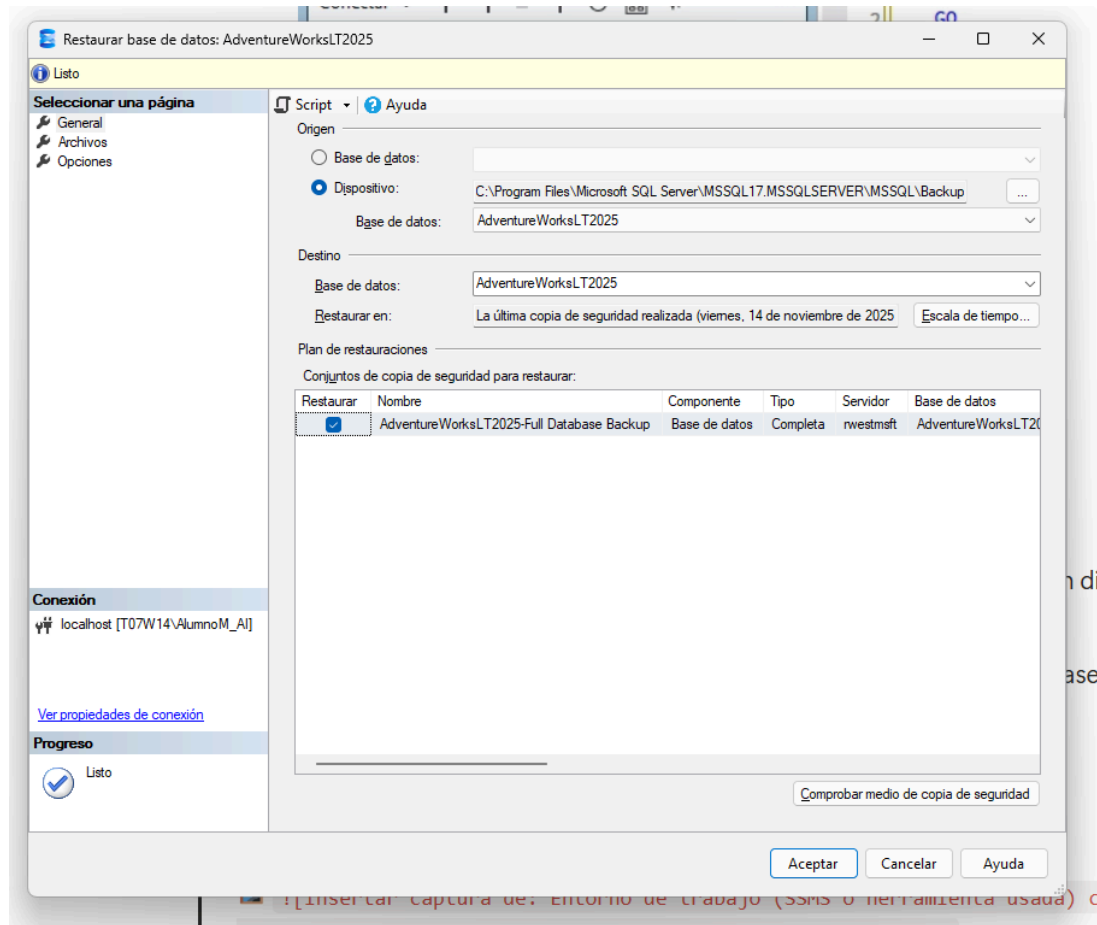
- Una herramienta de consultas como SQL Server Management Studio.
- Una conexión con permisos `CREATE`.
- Base de datos de ejemplo AdventureWorksLT (SQL Server o Azure SQL).
- necesitamos hacer un restorage de nuestros datos de la base de datos que importaremos un backup oficial
 - Descargamos de la página oficial el `.bak` la versión del SQL que tengas y LT versión
 - Movemos el archivo a la ruta `C:\Program Files\Microsoft SQL Server\MSSQL17.MSSQLSERVER\MSSQL\Backup`
 - Abrimos el SSMS
 - Haz clic **derecho en Bases de Datos** en el **Explorador de Objetos** y luego **selecciona Restaurar Base de Datos...** para iniciar el asistente **Restaurar Base de Datos**.



- **Selecciona Dispositivo** y luego selecciona la puntería (...) para elegir un dispositivo.
- Ahora dale a añadir en la Interfaz en la ruta donde alojaste tu `.bak`
- Selecciona **OK** para confirmar tu selección de copia de seguridad de base de datos y cierra la ventana **Seleccionar dispositivos de copia de**

seguridad

- Antes de continuar revisa los archivos y ya luego dale a OK



2. Desarrollo

2.1 Conexión a AdventureWorksLT

Verificación de la conectividad y de las tablas clave (`SalesLT.Customer` , `SalesLT.SalesOrderHeader` , `SalesLT.Product`).

```
-- Verify key tables in AdventureWorksLT
SELECT TOP (5) CustomerID, FirstName, LastName
FROM SalesLT.Customer;

SELECT TOP (5) SalesOrderID, OrderDate, CustomerID
FROM SalesLT.SalesOrderHeader;
```

```
SELECT TOP (5) ProductID, Name, ListPrice
FROM SalesLT.Product;
```

Nos devuelve correctamente el top de los datos de las tres tablas seleccionadas en cada consulta

The screenshot shows a SQL Server Enterprise Manager window with three queries. The first query is a comment: `-- Verify key tables in AdventureWorksLT`. The second query is `SELECT TOP (5) CustomerID, FirstName, LastName FROM SalesLT.Customer;`. The third query is `SELECT TOP (5) SalesOrderID, OrderDate, CustomerID FROM SalesLT.SalesOrderHeader;`. The fourth query is `SELECT TOP (5) ProductID, Name, ListPrice FROM SalesLT.Product;`. The results are displayed in three tables below the queries.

CustomerID	FirstName	LastName
1	Orlando	Gee
2	Keith	Harris
3	Donna	Carreras
4	Janet	Gates
5	Lucy	Harrington

SalesOrderID	OrderDate	CustomerID
1	2008-06-01 00:00:00.000	29847
2	2008-06-01 00:00:00.000	30072
3	2008-06-01 00:00:00.000	30113
4	2008-06-01 00:00:00.000	29485
5	2008-06-01 00:00:00.000	29957

ProductID	Name	ListPrice
1	HL Road Frame - Black, 58	1431.50
2	HL Road Frame - Red, 58	1431.50
3	Sport-100 Helmet, Red	34.99
4	Sport-100 Helmet, Black	34.99
5	Mountain Bike Socks, M	9.50

2.2 Creación de una vista para simplificar consultas

Vista que combina clientes y sus pedidos en AdventureWorksLT (esquema `SalesLT`), ocultando la complejidad del `JOIN` al código de la aplicación.

```
CREATE OR ALTER VIEW SalesLT.vCustomerOrders AS
SELECT
    c.CustomerID,
    CONCAT(c.FirstName, ' ', c.LastName) AS CustomerName,
    h.SalesOrderID,
```

```

        h.OrderDate
FROM SalesLT.Customer c
INNER JOIN SalesLT.SalesOrderHeader h ON c.CustomerID = h.CustomerID;

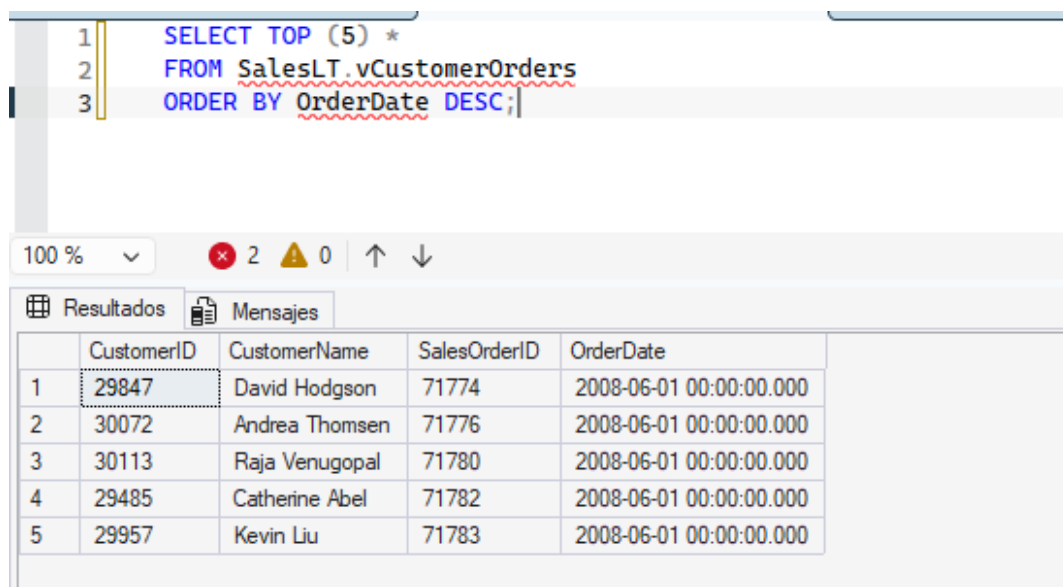
```

Validación de la vista:

```

SELECT TOP (5) *
FROM SalesLT.vCustomerOrders
ORDER BY OrderDate DESC;

```



The screenshot shows a SQL Server Enterprise Manager interface. The top pane displays a query window with the following SQL statement:

```

1  SELECT TOP (5) *
2  FROM SalesLT.vCustomerOrders
3  ORDER BY OrderDate DESC;

```

The bottom pane shows the results of the query in a table format. The table has five columns: CustomerID, CustomerName, SalesOrderID, and OrderDate. The results are ordered by OrderDate in descending order, showing the top 5 records.

	CustomerID	CustomerName	SalesOrderID	OrderDate
1	29847	David Hodgson	71774	2008-06-01 00:00:00.000
2	30072	Andrea Thomsen	71776	2008-06-01 00:00:00.000
3	30113	Raja Venugopal	71780	2008-06-01 00:00:00.000
4	29485	Catherine Abel	71782	2008-06-01 00:00:00.000
5	29957	Kevin Liu	71783	2008-06-01 00:00:00.000

2.3 Creación de un procedimiento almacenado para procesar un pedido

Procedimiento que añade una línea de pedido a un pedido existente en AdventureWorksLT y actualiza el subtotal de la cabecera.

```

CREATE OR ALTER PROCEDURE dbo.AddOrderLineItem
    @SalesOrderID INT,
    @ProductID INT,
    @Quantity INT
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRANSACTION;

```

```

-- Use Product ListPrice as UnitPrice
DECLARE @UnitPrice DECIMAL(18,2);
SELECT @UnitPrice = CAST(ListPrice AS DECIMAL(18,2))
FROM SalesLT.Product
WHERE ProductID = @ProductID;

IF @UnitPrice IS NULL
BEGIN
    ROLLBACK TRANSACTION;
    THROW 50010, 'Invalid ProductID specified.', 1;
END

-- Ensure SalesOrderID exists
IF NOT EXISTS (SELECT 1 FROM SalesLT.SalesOrderHeader W
HERE SalesOrderID = @SalesOrderID)
BEGIN
    ROLLBACK TRANSACTION;
    THROW 50011, 'Invalid SalesOrderID specified.', 1;
END

-- Insert line item (no discount)
INSERT INTO SalesLT.SalesOrderDetail (SalesOrderID, OrderQty, ProductID, UnitPrice, UnitPriceDiscount)
VALUES (@SalesOrderID, @Quantity, @ProductID, @UnitPrice, 0);

-- Update header subtotal based on current line totals
UPDATE h
SET SubTotal = d.SumLineTotal,
    ModifiedDate = SYSUTCDATETIME()
FROM SalesLT.SalesOrderHeader h
INNER JOIN (
    SELECT SalesOrderID, SUM(LineTotal) AS SumLineTotal
    FROM SalesLT.SalesOrderDetail
    WHERE SalesOrderID = @SalesOrderID
    GROUP BY SalesOrderID
) d ON d.SalesOrderID = h.SalesOrderID;

```

```
COMMIT TRANSACTION;  
END;
```

```
1 CREATE OR ALTER PROCEDURE dbo.AddOrderLineItem  
2     @SalesOrderID INT,  
3     @ProductID INT,  
4     @Quantity INT  
5 AS  
6 BEGIN  
7     SET NOCOUNT ON;  
8     BEGIN TRANSACTION;  
9  
10    -- Use Product ListPrice as UnitPrice  
11    DECLARE @UnitPrice DECIMAL(18,2);  
12    SELECT @UnitPrice = CAST(ListPrice AS DECIMAL(18,2))  
13    FROM SalesLT.Product  
14    WHERE ProductID = @ProductID;  
15  
16    IF @UnitPrice IS NULL  
17    BEGIN  
18        ROLLBACK TRANSACTION;  
19        THROW 50010, 'Invalid ProductID specified.', 1;  
20    END  
21  
22    -- Ensure SalesOrderID exists  
23    IF NOT EXISTS (SELECT 1 FROM SalesLT.SalesOrderHeader WHERE SalesOrderID = @SalesOrderID)  
24    BEGIN  
25        ROLLBACK TRANSACTION;  
26        THROW 50011, 'Invalid SalesOrderID specified.', 1;  
27    END  
28  
29    -- Insert line item (no discount)  
30    INSERT INTO SalesLT.SalesOrderDetail (SalesOrderID, OrderQty, ProductID, UnitPrice, UnitPriceDiscount)  
31    VALUES (@SalesOrderID, @Quantity, @ProductID, @UnitPrice, 0);  
32  
33    COMMIT TRANSACTION;  
34 END
```

100 %
No se encontraron problemas.
Línea: 46, Carácter: 6 SPC CRLF UTF-8

Mensajes

Los comandos se han completado correctamente.

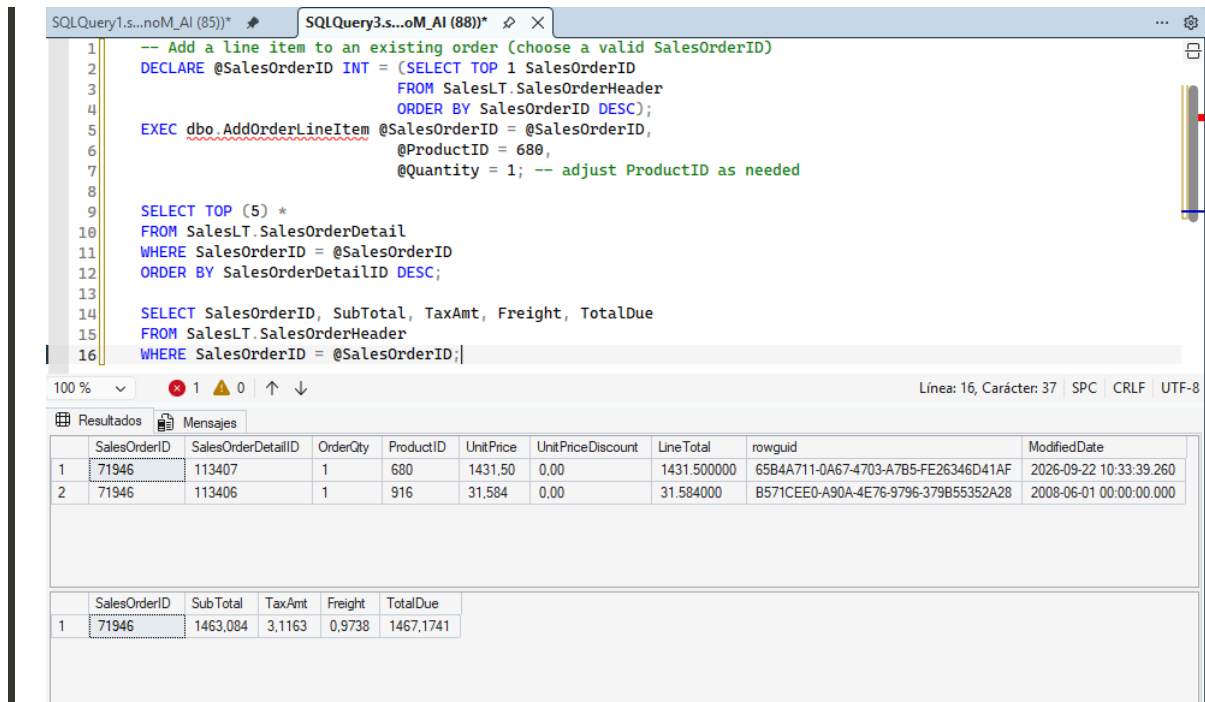
Hora de finalización: 2026-09-22T10:32:14.9203821+02:00

Prueba del procedimiento almacenado:

```
-- Add a line item to an existing order (choose a valid SalesOrderID)  
DECLARE @SalesOrderID INT = (SELECT TOP 1 SalesOrderID  
    FROM SalesLT.SalesOrderHeader  
    ORDER BY SalesOrderID DESC);  
EXEC dbo.AddOrderLineItem @SalesOrderID = @SalesOrderID,  
    @ProductID = 680,  
    @Quantity = 1; -- adjust ProductID as needed  
  
SELECT TOP (5) *  
FROM SalesLT.SalesOrderDetail  
WHERE SalesOrderID = @SalesOrderID
```

```
ORDER BY SalesOrderDetailID DESC;
```

```
SELECT SalesOrderID, SubTotal, TaxAmt, Freight, TotalDue  
FROM SalesLT.SalesOrderHeader  
WHERE SalesOrderID = @SalesOrderID;
```



The screenshot shows a SQL Server Enterprise Manager window with a query titled 'SQLQuery3.s...oM_AI (88)*'. The query is as follows:

```
1  -- Add a line item to an existing order (choose a valid SalesOrderID)  
2  DECLARE @SalesOrderID INT = (SELECT TOP 1 SalesOrderID  
3                                FROM SalesLT.SalesOrderHeader  
4                                ORDER BY SalesOrderID DESC);  
5  EXEC dbo.AddOrderLineItem @SalesOrderID = @SalesOrderID,  
6                             @ProductID = 680,  
7                             @Quantity = 1; -- adjust ProductID as needed  
8  
9  SELECT TOP (5) *  
10 FROM SalesLT.SalesOrderDetail  
11 WHERE SalesOrderID = @SalesOrderID  
12 ORDER BY SalesOrderDetailID DESC;  
13  
14 SELECT SalesOrderID, SubTotal, TaxAmt, Freight, TotalDue  
15 FROM SalesLT.SalesOrderHeader  
16 WHERE SalesOrderID = @SalesOrderID;
```

The results pane shows two tables. The first table has columns: SalesOrderID, SalesOrderDetailID, OrderQty, ProductID, UnitPrice, UnitPriceDiscount, LineTotal, rowguid, and ModifiedDate. It contains two rows of data.

SalesOrderID	SalesOrderDetailID	OrderQty	ProductID	UnitPrice	UnitPriceDiscount	LineTotal	rowguid	ModifiedDate
71946	113407	1	680	1431.50	0.00	1431.500000	65B4A711-0A67-4703-A7B5-FE26346D41AF	2026-09-22 10:33:39.260
71946	113406	1	916	31.584	0.00	31.584000	B571CEE0-A90A-4E76-9796-379B55352A28	2008-06-01 00:00:00.000

The second table has columns: SalesOrderID, SubTotal, TaxAmt, Freight, and TotalDue. It contains one row of data.

SalesOrderID	SubTotal	TaxAmt	Freight	TotalDue
71946	1463.084	3.1163	0.9738	1467.1741

2.4 Creación de una función escalar para cálculos reutilizables

Función escalar que devuelve el valor total de un pedido a partir de los importes de línea de AdventureWorksLT.

```
CREATE OR ALTER FUNCTION dbo.fnOrderTotal (@OrderID INT)  
RETURNS DECIMAL(18,2)  
AS  
BEGIN  
    DECLARE @Total DECIMAL(18,2);  
  
    SELECT @Total = SUM(LineTotal)  
    FROM SalesLT.SalesOrderDetail  
    WHERE SalesOrderID = @OrderID;
```

```
RETURN ISNULL(@Total, 0.00);
END;
```

The screenshot shows a SQL query window with the following code:

```
1 CREATE OR ALTER FUNCTION dbo.fnOrderTotal (@OrderID INT)
2 RETURNS DECIMAL(18,2)
3 AS
4 BEGIN
5     DECLARE @Total DECIMAL(18,2);
6
7     SELECT @Total = SUM(LineTotal)
8     FROM SalesLT.SalesOrderDetail
9     WHERE SalesOrderID = @OrderID;
10
11     RETURN ISNULL(@Total, 0.00);
12 END;
```

Below the code, a status bar indicates "No se encontraron problemas." (No problems found). A messages pane at the bottom shows the message: "Los comandos se han completado correctamente." (The commands have been completed successfully).

Uso de la función:

```
SELECT d.SalesOrderID, dbo.fnOrderTotal(d.SalesOrderID) AS
OrderTotal
FROM SalesLT.SalesOrderDetail d
GROUP BY d.SalesOrderID
ORDER BY d.SalesOrderID DESC;
```

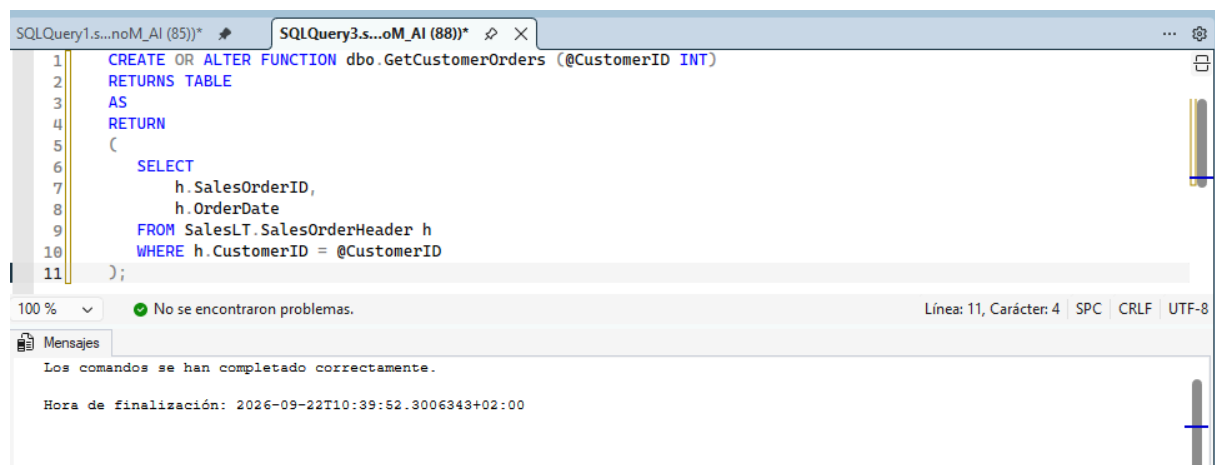
The screenshot shows the results of the query in a table with two columns: SalesOrderID and OrderTotal. The results are ordered by SalesOrderID in descending order.

	SalesOrderID	OrderTotal
1	71946	1463.08
2	71938	74160.23
3	71936	79589.62
4	71935	5533.87
5	71923	96.11
6	71920	2527.13
7	71917	37.76
8	71915	1732.89
9	71902	59894.21
10	71899	1856.21
11	71898	53248.69
12	71897	10585.05
13	71895	221.26
14	71885	524.66
15	71867	858.90
16	71863	2777.14

2.5 Creación de una función de tabla en línea (TVF)

TVF que devuelve los pedidos de un cliente concreto de AdventureWorksLT, útil en cláusulas `SELECT` y `JOIN`.

```
CREATE OR ALTER FUNCTION dbo.GetCustomerOrders (@CustomerID
INT)
RETURNS TABLE
AS
RETURN
(
    SELECT
        h.SalesOrderID,
        h.OrderDate
    FROM SalesLT.SalesOrderHeader h
    WHERE h.CustomerID = @CustomerID
);
```



Consulta directa de la función:

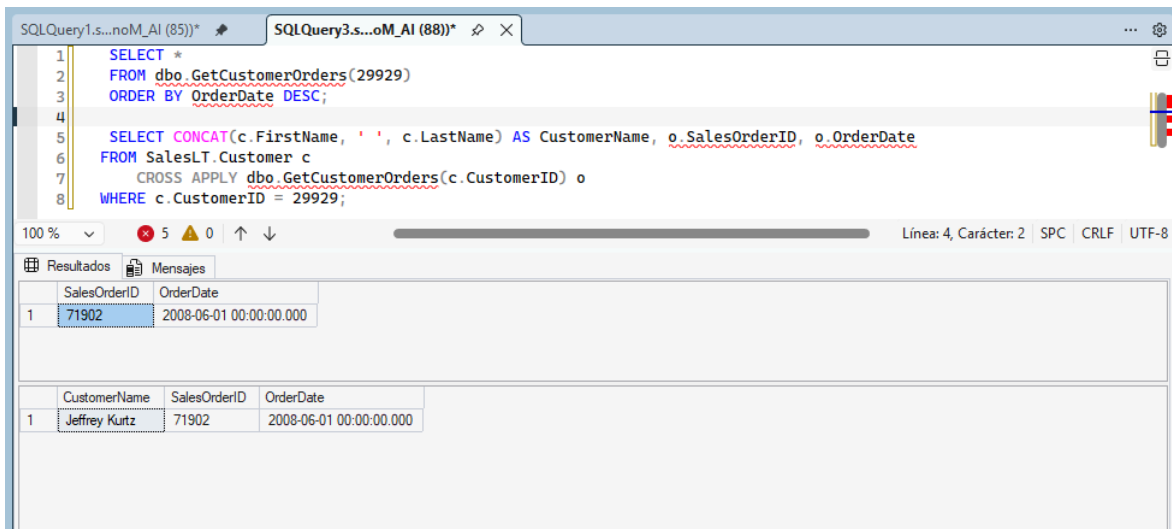
```
SELECT *
FROM dbo.GetCustomerOrders(29929)
ORDER BY OrderDate DESC;
```

Uso de la función junto con `CROSS APPLY`:

```
SELECT CONCAT(c.FirstName, ' ', c.LastName) AS CustomerName, o.SalesOrderID, o.OrderDate
```

```
FROM SalesLT.Customer c
      CROSS APPLY dbo.GetCustomerOrders(c.CustomerID) o
WHERE c.CustomerID = 29929;
```

Comprobación de la



The screenshot shows a SQL query window with the following code:

```
1 SELECT *
2 FROM dbo.GetCustomerOrders(29929)
3 ORDER BY OrderDate DESC;
4
5 SELECT CONCAT(c.FirstName, ' ', c.LastName) AS CustomerName, o.SalesOrderID, o.OrderDate
6 FROM SalesLT.Customer c
7      CROSS APPLY dbo.GetCustomerOrders(c.CustomerID) o
8 WHERE c.CustomerID = 29929;
```

The results pane displays two tables:

	SalesOrderID	OrderDate
1	71902	2008-06-01 00:00:00.000

	CustomerName	SalesOrderID	OrderDate
1	Jeffrey Kurtz	71902	2008-06-01 00:00:00.000

2.6 Creación de un trigger para registrar cambios

Trigger que registra actualizaciones del total de un pedido cuando cambian los detalles de pedido en `SalesLT`.

```
-- Audit table
IF OBJECT_ID('dbo.OrderAudit') IS NULL
BEGIN
    CREATE TABLE dbo.OrderAudit (
        AuditID      INT IDENTITY(1,1) PRIMARY KEY,
        OrderID      INT NOT NULL,
        OldTotal     DECIMAL(18,2) NULL,
        NewTotal     DECIMAL(18,2) NULL,
        ChangedAt    DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
    );
END
GO

-- Trigger on order details updates
CREATE OR ALTER TRIGGER SalesLT.trg_LogOrderTotalChange
```

```

ON SalesLT.SalesOrderDetail
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    ;WITH AffectedOrders AS (
        SELECT SalesOrderID FROM inserted
        UNION
        SELECT SalesOrderID FROM deleted
    ),
    -- New totals from the base table (already reflects changes)
    NewTotals AS (
        SELECT d.SalesOrderID, SUM(d.OrderQty * d.UnitPrice) AS Total
        FROM SalesLT.SalesOrderDetail d
        INNER JOIN AffectedOrders a ON d.SalesOrderID = a.SalesOrderID
        GROUP BY d.SalesOrderID
    ),
    -- Contribution of the newly inserted/updated rows
    InsertedTotals AS (
        SELECT SalesOrderID, SUM(OrderQty * UnitPrice) AS Total
        FROM inserted
        GROUP BY SalesOrderID
    ),
    -- Contribution of the previous row versions (empty on INSERT)
    DeletedTotals AS (
        SELECT SalesOrderID, SUM(OrderQty * UnitPrice) AS Total
        FROM deleted
        GROUP BY SalesOrderID
    )
    INSERT INTO dbo.OrderAudit (OrderID, OldTotal, NewTotal)

```

```

SELECT
    n.SalesOrderID,
    n.Total - ISNULL(i.Total, 0) + ISNULL(d.Total, 0) AS OldTotal,
    n.Total AS NewTotal
FROM NewTotals n
LEFT JOIN InsertedTotals i ON n.SalesOrderID = i.SalesOrderID
LEFT JOIN DeletedTotals d ON n.SalesOrderID = d.SalesOrderID;
END;

```

```

1  -- Audit table
2  IF OBJECT_ID('dbo.OrderAudit') IS NULL
3  BEGIN
4      CREATE TABLE dbo.OrderAudit (
5          AuditID      INT IDENTITY(1,1) PRIMARY KEY,
6          OrderID      INT NOT NULL,
7          OldTotal     DECIMAL(18,2) NULL,
8          NewTotal     DECIMAL(18,2) NULL,
9          ChangedAt    DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
10     );
11 END
12 GO
13
14 -- Trigger on order details updates
15 CREATE OR ALTER TRIGGER SalesLT.trg_LogOrderTotalChange
16 ON SalesLT.SalesOrderDetail
17 AFTER INSERT, UPDATE
18 AS
19 BEGIN
20     SET NOCOUNT ON;

```

100 % No se encontraron problemas. Línea: 54, Carácter: 6 SPC CRLF UTF-8

Mensajes

Los comandos se han completado correctamente.

Hora de finalización: 2026-09-22T10:43:54.0445383+02:00

Prueba del trigger:

```

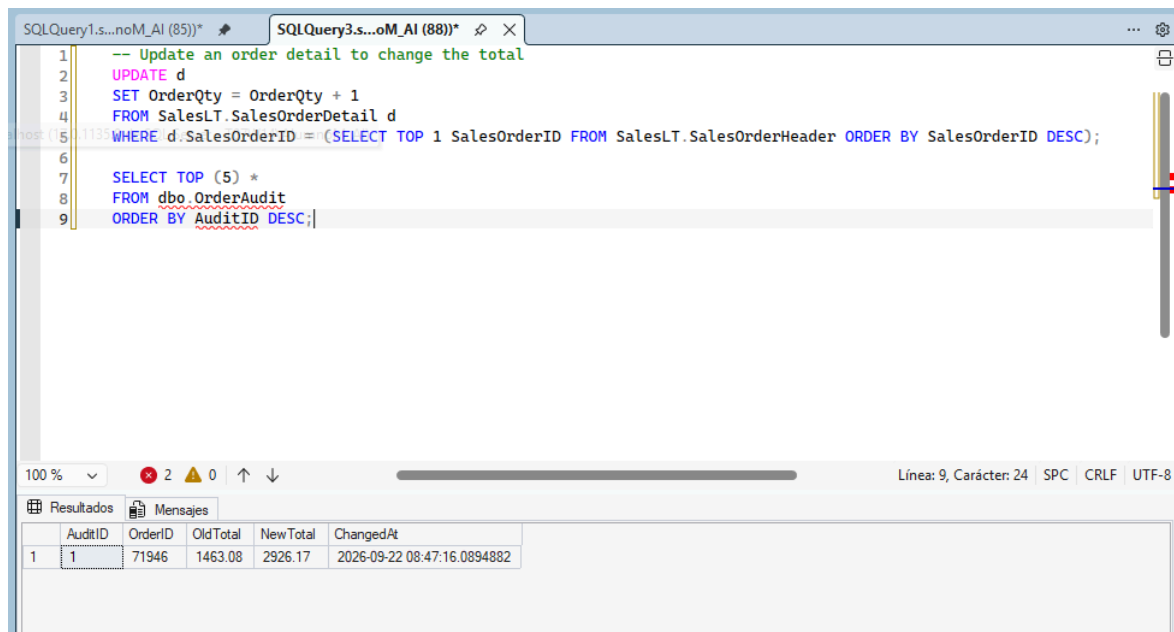
-- Update an order detail to change the total
UPDATE d
SET OrderQty = OrderQty + 1
FROM SalesLT.SalesOrderDetail d
WHERE d.SalesOrderID = (SELECT TOP 1 SalesOrderID FROM SalesLT.SalesOrderHeader ORDER BY SalesOrderID DESC);

SELECT TOP (5) *

```

```
FROM dbo.OrderAudit  
ORDER BY AuditID DESC;
```

El extracto devuelve las filas recientes de la tabla de auditoría. Cada fila muestra el que fue afectado, el total anterior (), el nuevo total () tras el cambio de cantidad y una marca de tiempo. Esto confirma que el disparador registró automáticamente la modificación. `SELECT OrderID OldTotal NewTotal`



3. Limpieza del entorno

No se realiza limpieza del entorno puesto a que se necesitara la base de datos para las siguientes partes del laboratorio

4. Resultados

A lo largo del laboratorio se han realizado las consultas propuestas sobre la base de datos `AdventureWorksLT 2025` de manera exitosa

5. Referencias

- [Implement programmability objects with SQL – Lab Exercises \(Microsoft Learn\)](#).
- [AdventureWorks sample databases – Microsoft Learn](#)